

增強課程 5：建立 Enhancement Spot 與 BAdI Definition

Lecture

en03 學過 BAdI 是「呼叫既有標準 BAdI」，這題要反過來：自己定義一個全新的 **BAdI**，讓別人（或未來的自己）可以掛 **Implementation** 上去。這是 en06（實作 BAdI）的前置準備，也是本課程第一次「從無到有」建立一個可擴充點，而不是查證/呼叫既有物件。

建立流程分兩層，且都是 **GUI-only**：

1. **BAdI Interface** (**INTF/OI**)：一個繼承 **IF_BADI_INTERFACE** (純標記介面，沒有任何方法，純粹讓 Kernel 認得這是合法的 BAdI Interface) 的 Interface，裡面宣告 BAdI 方法。這一步可以用 **ADT** 正常建立 (跟一般 Interface 建立沒有差異)。
2. **Enhancement Spot + BAdI Definition** (**ENHS/XS**)：這一步沒有 **ADT** 建立 **API**。實測直接 POST `/sap/bc/adt/enhancements/enhsxs` 不管怎麼調整 XML 結構 (依錯誤訊息陸續補上 `enhs:contentCommon`、`enhs:documentationId` 等)，最後都停在後端 kernel 層級的 **ASSERTION_FAILED** (不是 schema 驗證錯誤，是更根本的失敗)——判斷 Enhancement Spot 的建立必須走 **SE18**，跟 en04 的 **ENHOXHH** (Source Code Plugin) 建立需要 GUI 是同一類限制。

SE18 操作重點：Enhancement Spot 底下用「Add BAdI」新增一個 BAdI Definition (一個 Enhancement Spot 可以放多個 BAdI Definition，這題只用一個)，畫面上有幾個關鍵設定：

- **Interface**：填已經用 ADT 建好的 Interface 名稱
- **Multiple Use**：勾起來就是 Multi Use (可以有多個 Implementation 並存)，不勾就是 Single Use (系統最多一個 Implementation)
- **Fallback Class**：勾選「Call fallback if no implementation is executed」後指定一個實作該 Interface 的 Class，沒有任何 Active Implementation 時系統會呼叫它，提供預設行為

⚠ **Multi Use** 的關鍵限制：**Interface** 方法不能有 **RETURNING/EXPORTING** 參數——實測在 SE18 填完 Interface 後直接報錯：

```
Interface ZIF_EN05_FLIGHT_GREETING cannot be used (see long text)
The BAdI ZES_EN05_GREETING is not a single-use BAdI. However, the interface
ZIF_EN05_FLIGHT_GREETING has methods with returning or exporting parameters.
```

原因：Multi Use 可能同時有多個 Implementation 依序執行，「哪一個的回傳值算數」沒有明確定義，所以系統乾脆禁止 **RETURNING/EXPORTING**；資料流向要靠 **CHANGING** 參數——每個 Implementation 依序拿到、依序修改同一份資料，這是 Multi Use BAdI 資料流的標準設計模式。修法：把原本 **RETURNING VALUE(rv_text) TYPE string** 改成 **CHANGING cv_text TYPE string**。

Fallback Class 也可以順便登記成「**Implementation Example Class**」：SE18 填完 Fallback Class 後會跳出「Create fallback class as implementation example class as well?」——這是兩件獨立的事：**Fallback Class** 沒有 Implementation 時真的會被呼叫 (功能性)；**Implementation Example Class** 只是給之後建 Implementation 的人參考複製的範本 (純文件性質，不影響任何執行邏輯)。選 Yes 沒有副作用。

這題最大的篇幅在排錯 **GET BADI** 語法，過程值得完整記錄：

寫呼叫端驗證程式時，**GET BADI go_badi**。編譯期直接報錯 **"GO_BADI" is not a valid BAdI handle here**。——先懷疑是不是新物件的中繼資料還沒同步，做了對照測試：**連系統裡存在多年的真實標準 BAdI (MB_MIGO_BADI / IF_EX_MB_MIGO_BADI)** 用同樣語法測，一樣報同一個錯，而且在使用者自己的 SE38 GUI 裡也重現，排除了 ADT/RFC 連線快取的可能性。

改用 en03 驗證過的 Classic 語法 **cl_exithandler=>get_instance** 測試，又踩到第二個坑：**EXIT_NAME** 參數的型別 **EXIT_DEF** 是 **CHAR20**，我們原本取的 Enhancement Spot 名稱 **ZES_EN05_FLIGHT_GREETING** 有 24 字元，超過上限，編譯報「literal not type-compatible」——這是 **Classic Exit 命名慣例 (SMOD/CMOD 時代 20 碼上限)** 留下的硬限制，即使套用在新式 **BAdI** 名稱上一樣生效，改用 17 碼的 **ZES_EN05_GREETING** 才解決這一層。改名後 **cl_exithandler=>get_instance** 编译成功，但執行期又報 **DATA_INCONS_IN_EXIT_MANAGEM** (Exit 管理資料不一致) ——先後在「只有 Fallback Class、無 Implementation」與「已有真實 Implementation」兩種情境下都測過，結果一樣，證實跟有沒有 Implementation 無關。

真正的根因來自官方文件 (**sap-docs** MCP 查到的 ABAP Cheat Sheet 範例) 裡一個容易忽略的細節：

```
DATA badi_a TYPE REF TO zbadi_demo_abap_converter.    "← 這是 BAdI 名稱，不是
Interface !
GET BADI badi_a.
```

GET BADI 的參照變數型別要宣告成 **BAdI Definition** 本身的名稱 (**Kernel** 會自動生成一個同名、繼承 **CL_BADI_BASE** 的隱藏類別)，不是 **BAdI Interface** 的名稱——這正是官方文件另一段話的具體體現：「Classic BAdI 用工廠方法 + Interface 型別參照變數；Kernel-based BAdI 用 **GET BADI + BAdI** 型別參照變數」。我們先前兩次測試都寫成 **TYPE REF TO <interface名稱>** (包含對照測試用的標準 **IF_EX_MB_MIGO_BADI**)，這是全部失敗的共同原因，改成 **TYPE REF TO zes_en05_greeting** (BAdI 名稱本身) 後，**GET BADI/CALL BADI** 立刻正常編譯、正常執行。

這段排錯給的教訓比表面上的語法規則更重要：遇到「新物件不能用」的狀況，直覺會懷疑是不是物件本身沒設定好，但先用一個已知運作多年的真實標準物件做對照測試，能快速排除「是不是我的新物件有問題」這個變因，把懷疑範圍收斂到「呼叫方式本身」；接著去查官方文件的完整範例 (不是只看語法摘要)，逐字比對每一個型別宣告，往往能挖出這種容易被忽略的細節。

學習目標

- 能講出 Enhancement Spot + BAdI Definition 的建立流程：Interface 走 ADT、Enhancement Spot / BAdI Definition 走 SE18 (GUI-only，無 ADT 建立 API)
- 能講出 Multi Use BAdI 的 Interface 方法為什麼不能有 **RETURNING/EXPORTING**，資料要靠 **CHANGING** 參數傳遞
- 能講出 Fallback Class 的用途 (沒有 Active Implementation 時的預設行為) 與 Implementation Example Class 的差異 (前者功能性、後者純文件參考)
- 能正確宣告 **GET BADI** 的參照變數型別：要用 **BAdI 名稱本身**，不是 BAdI Interface 名稱——這是 Kernel-based BAdI 與 Classic BAdI (**cl_exithandler=>get_instance** 用 Interface 型別) 語法慣例的關鍵差異
- 知道 **cl_exithandler=>get_instance** 的 **EXIT_NAME** 參數型別 **EXIT_DEF** 只有 **CHAR20**，取 BAdI / Enhancement Spot 名稱時要留意這個舊時代遺留的長度限制

- 遇到「新建物件呼叫失敗」時，懂得用「已知運作正常的標準物件」做對照測試，快速判斷問題出在物件設定還是呼叫方式

事前準備 (已於本系統 client 130 實際完成，非假設)

1. **ZIF_EN05_FLIGHT_GREETING** (\$TMP，ADT 建立) : BAdI Interface，繼承 **IF_BADI_INTERFACE**，方法 **GET_GREETING(IMPORTING iv_carrid CHANGING cv_text)** (Multi Use 限制，改用 **CHANGING** 而非 **RETURNING**)。
2. **ZCL_EN05_FLIGHT_GREETING_FB** (\$TMP，ADT 建立) : Fallback Class，實作上述 Interface，回傳 **Welcome aboard <carrid>! (fallback greeting, no customer implementation active)**。
3. **ZES_EN05_GREETING** (\$TMP，使用者於 **SE18** 建立) : Enhancement Spot / BAdI Definition，Multi Use，Interface 為 **ZIF_EN05_FLIGHT_GREETING**，Fallback Class 為 **ZCL_EN05_FLIGHT_GREETING_FB** (同時登記為 Implementation Example Class)。注意：系統裡還留著一個命名過長 (24 碼)、從未真正成功呼叫過的 **ZES_EN05_FLIGHT_GREETING** 當反面教材，正式使用的是 17 碼的 **ZES_EN05_GREETING**。
4. **ZIM_EN05_GREETING / ZCL_EN05_GREETING** (\$TMP，使用者於 **SE19** 建立，Claude 用 ADT 寫入內容) : 真實 BAdI Implementation，回傳 **Bon voyage on <carrid>! (real implementation ZCL_EN05_GREETING is active)**，文字故意跟 Fallback Class 明顯不同，方便驗證呼叫的到底是哪一個。
5. **ZR_EN05_FLIGHT_GREETING_DEMO** (\$TMP) : 驗證程式，**DATA go_badi TYPE REF TO zes_en05_greeting. + GET BADI go_badi. + CALL BADI go_badi->get_greeting.** 已用 **programrun** 無頭執行，兩種情境皆驗證成功：
 - **Implementation** 啟用時：**CARRID=LH => Bon voyage on LH! (real implementation ZCL_EN05_GREETING is active)**
 - **Implementation** 停用時 (SE19 取消勾選「Implementation is active」) : **CARRID=LH => Welcome aboard LH! (fallback greeting, no customer implementation active)**
6. **ZR_EN05_CONTROL_TEST** (\$TMP) : 對照測試程式，用來排查 **GET BADI** 失敗原因，最終版本 **DATA go_badi TYPE REF TO zes_en05_greeting. / GET BADI go_badi.** 語法檢查通過，證實問題出在型別宣告方式而非物件設定。

題目需求

1. 畫出建立流程圖：Interface (ADT) → Enhancement Spot / BAdI Definition (SE18) → Implementation (SE19，en06 會做) 三個步驟，標出哪些走 ADT、哪些是 GUI-only。
2. 解釋 **Multi Use** 為什麼不能用 **RETURNING/EXPORTING**，並說明如果一定要讓每個 Implementation 都能「回報結果」，**CHANGING** 參數該怎麼設計 (提示：可以設計成一個內部表或結構，讓每個 Implementation 往裡面追加/合併自己的結果，而不是覆蓋前一個的輸出)。
3. **GET BADI** 型別宣告規則：解釋為什麼 **DATA go_badi TYPE REF TO zif_en05_flight_greeting.** (Interface 型別) 編譯失敗，**DATA go_badi TYPE REF TO zes_en05_greeting.** (BAdI 名稱型別) 才正確，這跟 **cl_exithandler=>get_instance** (Classic 語法，用 Interface 型別) 的宣告方式為什麼剛好相反？
4. 排錯方法論：這題排錯過程中，「用已知運作正常的標準 BAdI 做對照測試」這個步驟具體排除了什麼可能性？如果沒有做這個對照測試，直接去改自己新建的物件設定，會浪費多少時間在錯誤的方向上？
5. **Fallback Class vs Implementation Example Class**：兩者在 SE18 都會問到，差異是什麼？如果只想要「文件範本」不想要「真的有預設行為」，該怎麼設定？

參考答案

建立流程圖：

- ① BAdI Interface (ZIF_EN05_FLIGHT_GREETING)
 - └ ADT 直接建立，繼承 IF_BADI_INTERFACE

↓
- ② Enhancement Spot + BAdI Definition (ZES_EN05_GREETING)
 - └ SE18 GUI-only (無 ADT 建立 API，實測 POST 一律 kernel ASSERTION_FAILED)
 - └ 指定 Interface、Multi/Single Use、Fallback Class

↓
- ③ BAdI Implementation (ZIM_EN05_GREETING，en06 主題)
 - └ SE19 GUI-only 建立骨架，內容可以用 ADT 寫入

Multi Use 不能用 RETURNING/EXPORTING 的原因與替代設計：Multi Use 允許同時有多個 Implementation 並存執行，如果方法有 RETURNING/EXPORTING，多個 Implementation 依序執行時「最後回傳值該用哪一個」沒有明確定義（覆蓋？只取第一個？取最後一個？），系統乾脆從語言層級禁止，強迫用 CHANGING。如果需要「收集每個 Implementation 的結果」，CHANGING 參數可以設計成一個內部表（如 CHANGING ct_results TYPE string_table），每個 Implementation 收到的是同一份表，可以各自 APPEND 自己的結果進去而不是覆蓋，呼叫端最後拿到的是所有 Implementation 貢獻結果的集合。

GET BADI 型別宣告規則：Kernel-based BAdI 框架會在 BAdI Definition 啟用時，自動生成一個跟 BAdI 同名、繼承 CL_BADI_BASE 的隱藏類別，GET BADI 產生的物件實例就是這個隱藏類別的實例——所以參照變數要宣告成 TYPE REF TO <BAdI名稱>，型別系統才認得這是一個合法的「BAdI Handle」。

cl_exithandler=>get_instance 是更早期、Classic 世代的呼叫慣例，設計上是「拿到一個實作了某個 Interface 的物件」，所以用 TYPE REF TO <Interface名稱> / CHANGING instance TYPE ANY（泛型，執行期再做型別檢查）。兩者剛好相反，是兩個世代的 BAdI 存取機制在型別系統設計上的根本差異，混用會編譯失敗（GET BADI 用 Interface 型別）或執行期失敗（cl_exithandler 用 BAdI 名稱，會拿不到正確的 Exit 定義）。

排錯方法論：對照測試（拿 MB_MIGO_BADI 這個已知運作多年的真實標準 BAdI，用一模一樣的語法測）具體排除了「是不是我們新建的 Interface / Enhancement Spot / Fallback Class 少設定了什麼」這整條懷疑路徑——如果對照測試也失敗，代表問題不在新物件的設定，一定是呼叫語法本身或更底層的環境。如果沒做這一步，直接懷疑新物件設定不對，很可能會走進「反覆刪除重建 Interface / Enhancement Spot」「懷疑套件/傳輸設定」「懷疑角色權限」這幾類跟真正根因（型別宣告方式）完全無關的死路，浪費大量時間；本題實際上也是先繞了 cl_exithandler / EXIT_DEF 長度限制這條路才找到真正答案，但至少方向是收斂的（呼叫語法層級），不是發散地亂猜物件設定。

Fallback Class vs Implementation Example Class：Fallback Class 是功能性的——SE18 勾選「Call fallback if no implementation is executed」後，沒有任何 Active Implementation 時系統真的會呼叫它，實測驗證過（CARRID=LH => Welcome aboard LH!...）；Implementation Example Class 純粹是文件性質，SE19 建立新 Implementation 時可以參考複製這個範本，對任何實際執行邏輯沒有影響。如果只想要文件範本、不想要真的有預設行為：在 SE18 不要勾選「Call fallback if no implementation is executed」（或不填 Fallback Class 欄位），但仍然可以把某個 Class 標記為 Implementation Example Class——這兩個設定在 SE18 是分開的兩個獨立選項，可以只選其中一個。

思考題

1. 這題發現 Enhancement Spot / BAdI Definition 沒有 ADT 建立 API，但 en04 的 Source Code Plugin (**ENHOXHH**) 也是同樣的限制。如果之後 en06 (BAdI Implementation) 也發現同樣的模式，你會怎麼預測這個限制背後的原因？ (提示：這幾類物件都需要「在某個既有結構裡選擇一個位置/身分」—— Enhancement Spot 需要決定 Interface 綁定關係、Source Code Plugin 需要選擇隱式插入點位置、Implementation 需要綁定 Enhancement Spot——這種「選擇性綁定」的操作，SAP 傾向讓開發者在圖形化編輯器裡透過畫面互動完成，而不是開放成無狀態的 REST 建立 API)
2. **EXIT_DEF** 這個 CHAR20 限制，是 Classic Exit 時代 (**SMOD**) 留下的技術債，卻意外地也限制了新式 BAdI 在用 **cl_exithandler=>get_instance** 呼叫時的命名長度。如果一個團隊的命名慣例是「套件代碼 + 模組代碼 + 描述性名稱」，很容易超過 20 碼，你會建議這個團隊怎麼調整命名規則來避免這個坑？ (提示：可以只在「預期會用 **cl_exithandler** 呼叫的 BAdI」上額外套用一個更短的別名/縮寫慣例，或乾脆規定所有 BAdI / Enhancement Spot 名稱都不超過 20 碼，避免未來有人也用 Classic 語法呼叫時才發現這個限制)
3. 本題驗證程式在「Implementation 啟用」與「Implementation 停用」兩種狀態下重新執行過兩次，都拿到了正確、即時反映當下狀態的結果，沒有像 en04 學到的「GUI Session 快取舊版本」那樣的延遲現象。這兩種情境 (en04 的 Implicit Enhancement 程式碼變更 vs 這題的 Implementation 啟用/停用開關) 在底層機制上有什麼本質差異，導致一個會被快取、一個不會？ (提示：en04 改的是**程式碼本身**，程式碼變更需要重新產生 Load / Generated Code，已載入記憶體의 Session 不會自動重新產生；這題改的只是一個**布林開關 (是否啟用)**，BAdI 框架在每次 **GET BADI / CALL BADI** 當下都會即時查詢哪些 Implementation 是 Active 的，不涉及程式碼重新產生，所以能立即反映)